EXPRESSIONS, FUNCTIONS, AND CONTROL

COMPUTER SCIENCE MENTORS 61A

August 31, 2026 – September 4, 2026

1 Expressions

1. What Would Python Display?

```
>>> 3
```

```
>>> "cs61a"
```

```
>>> x = 3
>>> x
```

```
>>> x = print("cs61a")
cs61a
>>> x
```

```
>>> print(print(print("cs61a")))
```

```
>>> def f1(x):
...     return x + 1
>>> f1(3)
```

```
>>> f1(2) + f1(2 + 3)
```

```
>>> def f2(y):
...     return y / 0
>>> f2(4)
```

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

```
>>> def f3(x, y):
...     if x > y:
...             return x
...     elif x == y:
...             return x + y
...     else:
...             return y
>>> f3(1, 2)


>>> f3(5, 5)


>>> 1 or 2 or 3


>>> 1 or 0 or 3


>>> 4 and (2 or 1/0)


>>> 0 or (not 1 and 3)


>>> (2 or 1/0) and (False or (True and (0 or 1)))
```

2 Functions

1. For the following expressions, simplify the operands in the order of evaluation of the entire expression

   Example: `add(3, mul(4, 5))`

   Order of Evaluation: `add(3, mul(4, 5))` → `add(3, 20)` → `23`

   (a) `add(1, mul(2, 3))`

   (b) `add(mul(2, 3), add(1, 4))`

   (c) **`max`**`(mul(1, 2), add(5, 6), 3, mul(mul(3, 4), 1), 7)`

2. Sally really wants to know what she needs to score to pass her class! Fill out the function `score_needed` that takes in *current_grade*, her current grade, *num_exams*, the number of exams taken so far (not including her final exam), and *pass_grade*, the grade needed to pass. Assume that every exam, including the final exam, has the same weight.

```
def score_needed(current_grade, num_exams, pass_grade):
    '''
    >>> score_needed(89, 4, 90)
    94
    >>> score_needed(65, 2, 70)
    80
    >>> score_needed(77, 10, 78)
    88
    '''
```

3 Environment Diagrams

1. Give the environment diagram and console output that result from running the following code.

```
def swap(x, y):
    x, y = y, x
    return print("Swapped!", x, y)

x, y = 60, 1
a = swap(x, y)
swap(a, y)
```

4 Control

1. Write a function that returns `True` if a number is divisible by 4, 1 if a number is divisible by 7 and is not already divisible by 4, and returns `False` if neither condition is fulfilled.

```
def divisibility_check(num):
```

2. Implement `fizzbuzz(n)`, which prints numbers from 1 to `n` (inclusive). However, for numbers divisible by 3, print "fizz". For numbers divisible by 5, print "buzz". For numbers divisible by both 3 and 5, print "fizzbuzz".

```
def fizzbuzz(n):
    """
    >>> result = fizzbuzz(16)
    1
    2
    fizz
    4
    buzz
    fizz
    7
    8
    fizz
    buzz
    11
    fizz
    13
    14
    fizzbuzz
    16
    >>> result is None
    True
    """
```

5 Higher-Order Functions

1. Write a function, `whole_sum`, which takes in an integer, `n`. It returns another function which takes in an integer, and returns `True` if the digits of that integer sum to `n` and `False` otherwise.

```
def whole_sum(n):
    """
    >>> whole_sum(21)(777)
    True
    >>> whole_sum(142)(10010101010)
    False
    """
    def check(x):

        _____________________

        while _______________:

            last = ____________________

            ___________________________

            ___________________________

        return ____________________

    return ____________________
```